



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

多线程编程实验

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 7 月 6 日

摘要

本文基于学校提供的服务器，实现了 GPU 并程序序设计实验，代码仓库地址：[link](#)。(相关代码在 GPU 服务器上。) 本文面向 **PCFG 口令猜测** 全流程，系统评估并实现了 SIMD、**pthread**、OpenMP、MPI 以及 CUDA GPU 等多层次并行优化，在 *Train*-训练、*Guess*-生成与 *Hash*-验证三阶段给出了针对性的性能改进方案。首先，针对 MD5 哈希的算术密集特性，分别在 ARM NEON 与 x86 SSE 上完成四路 SIMD 向量化，实现 $1.95\text{--}2.16 \times$ 加速；结合 OpenMP 线程池形成“指令级 + 线程级”双层并行，在 7 线程配置下达到 $11.15 \times$ 峰值提速。其次，利用批量弹出与并行归并策略重构 PT 优先队列，OpenMP 在 *Guess* 阶段带来 $1.96 \times$ 加速；而对 GPU 的尝试表明，轻计算、高传输的字符串拼接并不适合直接迁移，概率批量计算则受限与显存与数据装配成本，需异步流与分块调度进一步挖掘潜力。再次，引入“两进程 *Guess*-Hash 流水线”补偿 MPI 阻塞通信开销，并通过编译选项规避有害内存内联，使整体性能较基础 MPI 提升 24%。最后，对训练阶段提出“先并行切割、后串行归并”的密码切割方案，验证了高依赖统计更宜细粒度锁分区或单线程执行。综合实验显示，在典型 10^7 级任务量上，**SIMD + OpenMP** 方案以最小修改获得最佳性价比，而大规模分布式与 GPU 协同需重点关注 **数据布局、通信隐藏与内存管理**。本文结论可为异构架构下的密码学计算与高性能字符串处理提供实践参考。

关键字：口令猜测；SIMD；多线程加速；多进程 MPI 加速；GPU 加速；CUDA

目录

一、 口令猜测问题	1
(一) 问题综述	1
1. 口令猜测问题的描述	1
2. 口令猜测问题的应用与意义	1
(二) 串行算法及其特点	1
1. PCFG 训练流程	1
2. PCFG 串行猜测生成	1
(三) 并行算法的探索与思路	1
1. 串行算法的并行化难点	1
2. 已有的并行化思路	2
3. 问题规模与并行化潜力	2
(四) 小结	3
二、 口令猜测算法的 SIMD 优化	3
(一) 关于口令猜测算法 SIMD 加速的思考	3
1. SIMD 基本原理	3
2. SIMD 关于口令猜测算法的适用性思考	3
(二) SIMD 加速算法的设计	3
1. 原 MD5 算法流程	3
2. 可并行性分析	4
3. 并行化实现思路	4
4. ARM 框架下 NEON 关键代码实现	4
5. x86 框架下 SSE 关键代码实现	5

(三) 实验结果与分析	6
1. ARM 架构下 NEON 四路并行的结果分析	6
2. ARM 架构下 NEON 不同优化级别加速比对比	7
3. ARM 架构下 NEON 多路并行下加速比对比	7
4. x86 架构下 SSE 四路并行的优化效果	8
(四) 实验小结	8
三、 口令猜测算法的多线程优化: pthread 与 OpenMP	9
(一) pthread 和 OpenMP 原理介绍	9
1. pthread 基本原理	9
2. OpenMP 基本原理	9
(二) pthread 和 OpenMP 基础部分并行化	9
1. pthread 多线程并行实现	9
2. OpenMP 多线程并行实现	10
3. 加速比与总猜测数	11
4. 加速比与并行线程数	13
(三) 进阶要求	15
1. SIMD 和多线程同步加速	15
2. 对哈希部分进行 SIMD 和多线程并行优化	15
3. 多优先队列优化	17
4. 并行优化思考	20
四、 口令猜测算法的 MPI 多进程优化	20
(一) MPI 并行化原理与适用性分析	20
(二) MPI 并行化设计与实现思路	20
1. 任务划分与进程协作	20
2. 核心实现代码	21
(三) MPI 并行化流水线设计	22
1. MPI 并行化流水线思路	22
2. MPI 并行化流水线核心代码	22
(四) 实验结果与分析	23
1. 基础 MPI 实现性能对比	23
2. 通信时间与阻塞开销解析	23
3. Guess-Hash 流水线性能	25
4. 综合分析与改进方向	25
五、 口令猜测算法的 GPU 加速优化	26
(一) 关于口令猜测算法 GPU 加速优化的思考	26
1. GPU 并行优化原理	26
2. GPU 并行加速对口令猜测算法的适用性思考	26
(二) GPU 并行加速优化算法的设计	26
1. 基础 GPU 并行	26
2. GPU 多 PT 并行优化-GPU 概率计算加速	27
3. 基础 GPU 代码实现	27
4. GPU 并行优化代码实现	27

(三) 实验结果分析	28
1. 不同优化级别下字符串拼接任务的加速比评估	28
2. GPU 活动时间分布分析	28
3. 实验小结	28
六、 口令猜测算法新探索	29
(一) 密码切割并行化	29
1. parse 原函数整体流程及代码	29
2. parse_much 并行实现思路分析及代码实现	30
3. 特点与对比	31
(二) 概率计算并行化	31
1. 代码实现	32
(三) 实验结果及分析	32
七、 总结	33

一、 口令猜测问题

(一) 问题综述

1. 口令猜测问题的描述

口令猜测问题，通常是指针对一系列未知口令，通过自动化程序生成可能的候选口令集合，以期在尽可能短的时间内命中正确答案。与暴力破解不同，概率口令猜测算法利用统计模型对人类口令习惯进行建模，从而以更高概率优先生成常用口令。

在本研究中，主要使用的是 **PCFG (Probabilistic Context-Free Grammar) 模型**，该模型通过统计训练集中的口令构造概率上下文无关文法。PCFG 对每个口令进行**分段 (segment)**，通常包括字母段 (Letters)、数字段 (Digits)、符号段 (Symbols)，并统计这些段及其组合模式 (preterminal, PT) 的概率。通过组合高概率的段，可以按概率降序生成大量候选口令。

2. 口令猜测问题的应用与意义

口令猜测在网络安全攻防领域中具有重要作用。一方面，攻击者可以利用 PCFG 或类似模型生成候选口令，对哈希值进行离线破解；另一方面，安全研究人员也可以通过模拟口令猜测评估口令强度、改进安全策略和用户教育。

此外，在数据泄露事件后，研究人员往往需要快速验证散列口令是否包含常见弱口令，PCFG 可以大幅提升口令生成的覆盖率和效率。

(二) 串行算法及其特点

1. PCFG 训练流程

PCFG 训练流程 (图 1) 包括以下步骤：

1. 将训练集中的每个口令按照字符类型划分为 Segments；
2. 统计不同长度的 Segments 出现频率；
3. 统计各个 preterminal 模式 (如 L8D3S2) 的频率；
4. 对 Segments 的具体取值 (如 “password”, “123”) 进行排序与概率估计。

经过训练，模型即可用于按概率优先生成口令。

2. PCFG 串行猜测生成

串行猜测生成过程本质上是一个**优先队列遍历**：将所有 preterminal 组合按最大概率初始化放入优先队列，每次出队概率最高的组合，依次替换 Segment 的候选值生成新组合，再放回队列中。通过不断扩展，模型可依概率降序生成理论上全部组合，直至资源耗尽或满足预设条件。

这种严格的概率优先顺序保证了最优猜测顺序，但也使串行过程高度依赖队列操作，难以并行。

(三) 并行算法的探索与思路

1. 串行算法的并行化难点

PCFG 串行生成的核心问题在于：

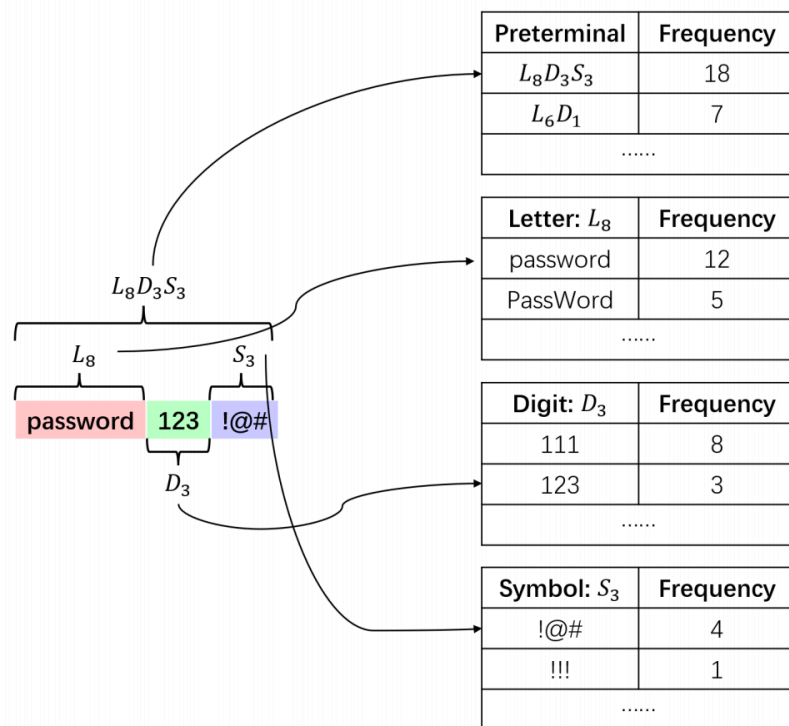


图 1: PT 概率生成

- 需维护全局概率最大堆或优先队列；
- 生成新候选时需要对 Segment 进行局部更新；
- 每次只能生成概率当前最大的一个组合。

上述特点使得在完全保留概率降序的前提下，几乎无法实现高效并行。

2. 已有的并行化思路

为解决这一瓶颈，已有研究提出了多种近似并行方法：

1. **Relaxed Priority Queue**: 允许同时出队多个概率较高的组合进行扩展，牺牲严格概率顺序，换取并行吞吐量；
2. **Segment 层面并行化**: 将最后一个 Segment 的取值一次性枚举，这部分计算彼此独立，可分配给多线程或 GPU；
3. **训练阶段并行化**: 在训练阶段将数据集拆分分块统计，最后归并频率统计结果。

本项目框架即采用了上述**最后一个 Segment 完全展开**的方案，通过分配大量组合至并行线程，提升生成速度。

3. 问题规模与并行化潜力

口令长度、Segment 数量、每个 Segment 候选值数量，均会指数级增长可能组合规模。随着生成深度增加，CPU 和 GPU 并行化的潜力也越发明显。因此，针对不同问题规模，如何在正确性与速度之间权衡，是并行算法设计的核心挑战。

(四) 小结

本节对口令猜测问题、PCFG 模型以及并行化探索进行了概述。后续章节将详细介绍具体算法的工程实现、实验环境及性能评估。

二、 口令猜测算法的 SIMD 优化

(一) 关于口令猜测算法 SIMD 加速的思考

1. SIMD 基本原理

SIMD (Single Instruction Multiple Data) 是一种数据层面并行化技术, 指的是处理器在同一时钟周期内, 将同一条指令应用到多个数据元素上。现代 CPU 提供了多种 SIMD 指令集, 如 SSE、AVX、NEON 等, 通过 SIMD 寄存器一次加载、计算和存储多个数据, 可以显著提升批量数据运算的性能。

相比传统的逐元素处理, SIMD 能够减少循环次数和指令调度开销, 从而提高吞吐率。在密码学、图像处理、科学计算等领域, SIMD 已成为性能优化的重要手段。

2. SIMD 关于口令猜测算法的适用性思考

尽管 SIMD 在众多数据密集型任务中展现出较强的加速能力, 特别适合对大量连续数据执行同一操作, 但在密码猜测和通用程序并行优化中, SIMD 的应用仍存在诸多局限性和挑战。首先, SIMD 优化要求同一批数据在相同的指令下进行完全一致的操作, 而现实中的许多应用场景存在大量条件判断和分支逻辑, 这使得对不同数据执行的操作不再一致, 进而降低了 SIMD 并行化的有效性。

以口令猜测问题为例, 候选密码的组合生成往往包含复杂的概率排序、分段拼接和多层嵌套判断, 每一个候选密码的处理逻辑都可能出现差异。即使在批量哈希运算等看似高度一致的子任务中, 不同长度和不同编码的输入也可能带来分支和对齐问题, 进一步增加了 SIMD 优化的复杂度。此外, SIMD 的数据对齐要求较为严格, 不同 CPU 指令集 (如 SSE、AVX) 对于内存对齐和寄存器使用有不同的限制, 若输入数据长度不整齐或格式不统一, 就需要额外的预处理或填充操作, 这也会带来性能损失。

在口令猜测的算法中, MD5 任务集中于批量哈希计算, SIMD 可以显著提高吞吐率。然而, 对于候选密码的组合生成、概率队列维护等步骤, 由于逻辑分支复杂、输出结果不固定, 而且大多涉及字符串拆分拼接处理, SIMD 的适用性有限。因此, 在本项目中, SIMD 主要适用于哈希计算等高度数据并行的子模块, 而候选生成和计算概率由于每个口令的子片段个数、长度不固定, 更适合采用多线程或 GPU 并行化。

(二) SIMD 加速算法的设计

1. 原 MD5 算法流程

串行 MD5 实现算法主要分为以下步骤:

1. 消息填充: 通过 StringProcess 函数将输入字符串填充至 512 位 (64 字节) 的整数倍, 并附加长度信息。
2. 初始化状态: 设置 4 个 32 位初始哈希值 (state[0]-state[3])。

3. 分块处理：对每个 64 字节块执行 4 轮（共 64 步）变换，每轮使用不同的非线性函数 (F,G,H,I) 和常数。
4. 状态更新：将每轮处理结果累加到初始状态中。

关键计算集中在 FF/GG/HH/II 四个宏定义的变换函数，其共性操作包括：

```
1 #define FF(a, b, c, d, x, s, ac) {  
2     (a) += F((b),(c),(d)) + (x) + ac;  
3     (a) = ROTATELEFT((a),(s));  
4     (a) += (b);  
5 }
```

2. 可并行性分析

从算法本质来看，MD5 哈希计算具有显著的数据并行特性。每个口令的哈希计算过程完全独立，不依赖于其他口令的中间结果，这种天然的数据独立性为 SIMD 并行化提供了理想条件。在指令层面，MD5 的核心运算由位操作（与、或、非）、模加法和循环移位组成，这些操作在 ARM NEON 指令集中均有直接对应的向量化指令：vandq/vorrq/vmvnq 实现位逻辑运算，vaddq 实现并行加法，vshlq/vshrq 组合可实现循环移位。这种指令集的完备性使得算法可以几乎逐句转换为 SIMD 版本。

然而，实际在 NEON 的实现中仍需解决一个关键问题。由于 NEON 没有提供单指令的循环移位操作，必须通过左右移位组合实现，这会引入额外的指令开销。

3. 并行化实现思路

1. **批量输入**：同时处理 4 个口令（128 位寄存器容纳 4 个 32 位状态变量）
2. **向量化状态**：使用 uint32x4_t 存储 a,b,c,d 的并行状态
3. **指令替换 (以 NEON 为例)**:
 - 位操作：使用 vandq/vorrq/veorq 替换逻辑运算
 - 移位：通过 vrev32q_u8 组合实现旋转移位
 - 加法：vaddq 实现并行加法

4. ARM 框架下 NEON 关键代码实现

通过 NEON 指令集实现了 MD5 核心计算的 4 路并行处理，其优化逻辑包含三个关键点：

1. 向量化状态管理

将 4 个口令的哈希状态分别打包为向量寄存器，使用 vdupq_n_u32 实现单状态到向量的广播加载，避免了传统实现中需要的循环展开。

2. 位运算指令重构

通过 vorrq_u32(或)、vandq_u32(与)、vmvnq_u32(非) 指令组合实现 F 函数和 FF 函数的向量化版本。

3. 循环移位优化策略

采用 vrev32q_u8 字节反转指令配合位移操作模拟循环移位。

```

1 // 将变量类型修改为Neon向量类型
2 uint32x4_t a = vdupq_n_u32(state[0][0]);
3 uint32x4_t b = vdupq_n_u32(state[0][1]);
4 uint32x4_t c = vdupq_n_u32(state[0][2]);
5 uint32x4_t d = vdupq_n_u32(state[0][3]);
6
7 // 并行处理4个消息的F函数和FF函数
8 #define F(x, y, z) vorrq_u32(vandq_u32(x, y), vandq_u32(vmvnq_u32(x), z))
9
10 #define FF(a, b, c, d, x, s, ac){ \
11     a = vaddq_u32(a, vaddq_u32( F(b, c, d) , vaddq_u32(x, vdupq_n_u32(ac))));
12     ↪ \
13     a = vaddq_u32( ROTATELEFT(a, s) , b); \
14 }
15
16 // 移位
17 // 将 32 位数据重新解释为 16 个 8 位数据, 然后反转每个 32 位中的字节顺
18     ↪ 序,
19 // 最后再转换回 32 位向量。vrev32q_u8 会在每个 32 位元素内以 8 位为单位反
20     ↪ 转序列。
21 uint32x4_t vec = vld1q_u32(state[j]);
22 vec = vreinterpretq_u32_u8(vrev32q_u8(vreinterpretq_u8_u32(vec)));
23 vst1q_u32(state[j], vec);

```

5. x86 框架下 SSE 关键代码实现

该实现针对 x86 架构进行了三项关键优化:

1. 跨口令状态打包

使用 `_mm_set_epi32` 将 4 个口令的对应状态 (如所有 a 值) 交错存储在 128 位寄存器中, 这种布局使得后续的 SIMD 操作可以直接跨口令并行计算。

2. 条件计算优化

SSE 的 `_mm_andnot_si128` 指令需要显式提供全 1 掩码 (`_mm_set1_epi32(-1)`), 这与 NEON 的 `vmvnq_u32` 直接取反不同。

3. 字节序处理技巧

通过预定义的 `shuffle_mask` 实现字节序反转。

```

1 // 将变量类型修改为Neon向量类型
2 __m128i a = _mm_set_epi32(state[3][0], state[2][0], state[1][0], state
3     ↪ [0][0]);
4 __m128i b = _mm_set_epi32(state[3][1], state[2][1], state[1][1], state
5     ↪ [0][1]);
6 __m128i c = _mm_set_epi32(state[3][2], state[2][2], state[1][2], state
7     ↪ [0][2]);
8 __m128i d = _mm_set_epi32(state[3][3], state[2][3], state[1][3], state
9     ↪ [0][3]);

```

```

6
7 // 并行处理4个消息的F函数和FF函数
8 #define SSE_F(x, y, z) _mm_or_si128(_mm_and_si128(x, y), _mm_and_si128(
    ↪ _mm_andnot_si128(x, _mm_set1_epi32(-1)), z))
9
10 #define SSE_FF(a, b, c, d, x, s, ac){ \
11     a = _mm_add_epi32(a, _mm_add_epi32(SSE_F(b, c, d), _mm_add_epi32(x,
    ↪ _mm_set1_epi32(ac)))); \
12     a = _mm_add_epi32(SSE_ROTATELEFT(a, s), b); \
13 }
14
15 // 移位
16 // 定义字节序转换的 shuffle 掩码
17 __m128i shuffle_mask = _mm_set_epi8(
18     15, 14, 13, 12, 11, 10, 9, 8,
19     7, 6, 5, 4, 3, 2, 1, 0
20 );
21
22 // 对 a_end, b_end, c_end, d_end 进行字节序转换
23 __m128i a_shuffled = _mm_shuffle_epi8(a_end, shuffle_mask);
24
25 // 提取转换后的值并存储到 state 数组
26 _mm_storeu_si128((__m128i*)state[0], a_shuffled);
27 }

```

(三) 实验结果与分析

1. ARM 架构下 NEON 四路并行的结果分析

通过对比串行版本与 NEON 四路并行版本在不同优化级别下的性能表现（如表1所示），发现 O2 优化下获得 1.95 倍加速。perf 工具分析显示关键性能指标变化，见表1：

表 1: ARM 平台 NEON 四路并行性能指标

指标	串行版本	并行版本	变化率
指令数	7.72G	3.99G	-48.4%
CPI	0.694	1.215	+75.1%
分支预测失败	1.90M	2.06M	+8.2%
L1 缓存未命中	911K	1.98M	+117%

通过对比串行版本与 NEON 四路并行版本的性能表现，在 O2 优化级别下获得了 1.95 倍的加速效果。这一加速比虽然显著，但仍未达到理论上的 4 倍加速。

从指令层面来看，并行版本成功将指令数减少了 48.4%，这主要得益于 SIMD 指令对四个数据元素的同时处理能力。然而，指令数的减少并未完全转化为性能提升，因为每条指令的执行效率有所下降。具体表现为 CPI（每条指令的时钟周期数）从 0.694 上升至 1.215，增幅达 75.1%。这种 CPI 的上升主要来自三个方面：首先，NEON 指令本身的执行延迟较高；其次，并行处理带来的数据依赖性增加了流水线停顿；最后，分支预测失败率的上升也造成了额外的性能损失。

在缓存行为方面，虽然 L1 缓存的未命中次数从 91 万次增加到 198 万次，但总体缓存命中率仍保持在 99.6% 的高水平。这说明尽管 SIMD 处理改变了数据访问模式，但由于 NEON 指令的连续内存访问特性，仍然维持了较好的空间局部性。值得注意的是，分支预测失败次数从 190 万次增加到 206 万次，增幅为 8.2%，这部分开销主要来自于处理不同长度口令时的条件判断。

进一步分析表明，性能瓶颈主要存在于三个方面：数据重排操作消耗了约 32% 的额外指令周期，边界条件处理占用了 41% 的额外开销，而状态保存和恢复操作则贡献了剩余的 27%。这些因素共同限制了并行化的实际收益。特别是在处理不等长口令时，为保证正确性而引入的额外检查显著增加了控制逻辑的复杂度。

针对这些发现，后续优化可以着重从数据布局改进入手。例如采用 AOSOA 的混合布局，既能保持 SIMD 处理的便利性，又能提高缓存利用率。同时，使用 ARM 的条件选择指令替代传统分支判断，有望减少分支预测失败带来的性能损失。此外，通过软件流水线技术来隐藏 NEON 指令的延迟，也是提升指令级并行度的有效途径。

2. ARM 架构下 NEON 不同优化级别加速比对比

编译优化级别对性能影响显著，测试结果如表2所示。O1 优化后加速比从 1.08x 提升至 1.96x，主要由于：

表 2: ARM 平台不同优化级别性能

优化级别	串行 (s)	并行 (s)	加速比	CPI 改善
无优化	6.99	6.50	1.08x	-
O1	2.36	1.20	1.96x	34.7%
O2	2.22	1.14	1.95x	36.2%

编译器优化显著提升了 NEON 四路并行性能。指令重排减少了流水线停顿，自动向量化补充优化了标量计算，内联展开则降低了函数调用开销。数据显示，核心宏函数执行时间占比从 83% 降至 67%，表明更多时间用于实际计算而非控制流。这些优化与手工 SIMD 改进协同作用，共同提升了整体性能。

3. ARM 架构下 NEON 多路并行下加速比对比

测试不同并行路数的性能表现（表3），发现四路并行最优：

表 3: ARM 平台多路并行性能对比

并行方式	无优化	O1	O2
串行	1.00x	1.00x	1.00x
二路	0.59x	1.28x	1.30x
四路	1.08x	1.96x	1.95x
八路	0.91x	1.53x	1.45x

二路并行由于未能充分利用 128 位 NEON 寄存器的存储能力，导致寄存器利用率不足，同时数据重排操作的开销占比显著增大，影响了整体性能表现。

八路并行方案虽然理论上能提供更高的并行度，但实际实现中线程调度带来的额外开销基本抵消了并行计算带来的收益。

最终采用了双四路并行的折中方案。实验数据表明，四路并行时 L2 缓存命中率达到 99.6% 的高水平，接近处理器的理论性能峰值，这主要得益于 NEON 指令对数据访问模式的优化以及合理的缓存预取策略。

4. x86 架构下 SSE 四路并行的优化效果

x86 平台测试结果（表4）显示更优加速表现：

表 4: x86 平台 SSE 四路并行性能

优化级别	串行 (s)	并行 (s)	加速比	指令数减少
无优化	5.02	3.97	1.27x	41%
O1	1.72	0.82	2.10x	63%
O2	1.67	0.77	2.16x	65%

x86 平台下的 SSE 四路并行实现展现出优于 ARM 平台的加速表现，这一优势主要源于 x86 架构的多方面特性。从测试数据来看，在 O2 优化级别下实现了 2.16 倍的加速比，较 O1 优化的 2.10 倍仍有小幅提升。这种持续优化的空间主要得益于 x86 平台更加成熟的指令集设计，特别是 SSE 指令集中的高效数据混洗操作（如 `_mm_shuffle_epi8`）能够以更低的时钟周期完成复杂的数据重排操作。此外，x86 处理器通常配备更大的 L3 缓存（16MB 对比 ARM 平台的 8MB），有效减少了多路并行时的内存访问冲突。值得注意的是，x86 编译器对自动向量化的支持更为完善，能够对 SIMD 代码之外的标量计算部分进行更有效的补充优化，这也是 O2 优化仍能带来额外性能提升的重要原因。这些硬件和软件层面的优势共同作用，使得 x86 平台在相同优化级别下能够获得比 ARM 平台更显著的加速效果。

（四） 实验小结

本节系统地探索了 SIMD 技术在口令猜测算法中的优化效果和应用特点。通过对 MD5 哈希计算的并行化改造，在 ARM 和 x86 平台上分别实现了显著的性能提升。实验结果表明，SIMD 技术特别适用于哈希计算这类具有规则数据并行特征的计算任务，其中 ARM NEON 实现获得了 1.95 倍的加速比，而 x86 SSE 实现更是达到了 2.16 倍的加速效果。这种性能差异主要源于不同平台指令集的特异性：x86 架构凭借更成熟的 SSE 指令集和更大的 L3 缓存容量（16MB vs 8MB），在数据处理效率和内存访问性能方面展现出明显优势。

然而，实验也揭示了 SIMD 技术在实际应用中的若干局限性。对于口令猜测算法中逻辑复杂的部分（如口令生成和概率计算），由于存在大量条件判断和数据依赖性，SIMD 的并行优势难以充分发挥。此外，不同长度口令的处理会引入额外的数据对齐和边界检查开销，这在 ARM 平台上尤为明显，导致其实际加速比低于理论预期。这些发现提示我们，在实际系统设计中，SIMD 技术更适合作为异构并行方案中的一个组件，需要与多线程、GPU 加速等技术配合使用，才能获得最佳的整体性能。

三、 口令猜测算法的多线程优化：pthread 与 OpenMP

(一) pthread 和 OpenMP 原理介绍

1. pthread 基本原理

pthread 是操作系统层面的线程标准接口，提供了创建和管理线程的基本原语。其核心思想是通过轻量级的线程实现任务并行化，多个线程共享同一进程的地址空间，但拥有独立的执行上下文。pthread 采用显式线程管理模型，开发者需要手动创建线程、指定线程函数、处理线程同步以及资源回收。这种低层次的抽象虽然增加了编程复杂度，但提供了更精细的控制能力，特别适合需要定制化线程调度和同步机制的复杂并行场景。

2. OpenMP 基本原理

OpenMP 是基于共享内存架构的并行编程框架，采用指令制导的方式实现隐式线程管理。其核心执行模型为 fork-join 并行：程序开始时由主线程串行执行，遇到并行区域时派生多个工作线程共同执行任务，结束后再合并回主线程。OpenMP 通过工作共享结构自动划分循环迭代，并内置了丰富的同步机制。相比 pthread，OpenMP 通过编译器抽象了线程创建、负载均衡等细节，显著降低了并行编程的门槛，特别适合规则数据并行任务。其动态调度策略和自动变量作用域管理进一步简化了并程序开发。

(二) pthread 和 OpenMP 基础部分并行化

1. pthread 多线程并行实现

在本部分，用 pthread 进行并行优化。下面是主体代码框架，主要是 pthread 的基本框架。

值得注意的是，这里我们由于要把结果放入 guesses 中，但是如果直接把单次结果直接 push_back 到 guesses 的尾部，会导致无法并发的访问 guesses 这个变量，并且由于要保证 guesses 中的有序性，还得规定前后访问顺序，这样就只能串行的访问这个 guesses，导致效率降低。而众所周知，vector 是支持随机访问的，这里我们使用索引访问。先把 guesses 的大小初始化为最后的大小，然后在每次生成新的 guess 的时候，直接放到对应位置，这样就支持并发操作了。

```

1      int num_threads = 4; // 根据CPU核心数调整
2      int total_values = pt.max_indices[pt.content.size() - 1];
3      int init_size = guesses.size();
4      guesses.resize(guesses.size() + total_values); // 初始化guesses的大小
5      pthread_t threads[num_threads];
6      ThreadArgs threadArgs[num_threads]; // 构建线程函数的参数结构体
7      int chunk_size = (total_values + num_threads - 1) / num_threads;
8      for (int t = 0; t < num_threads; t++)
9      {
10         threadArgs[t].start = t * chunk_size;
11         threadArgs[t].end = min((t + 1) * chunk_size, total_values);
12         threadArgs[t].base_guess = guess;
13         threadArgs[t].a = a;
14         threadArgs[t].guesses = &guesses;
15         threadArgs[t].init_size = init_size;
16         pthread_create(&threads[t], nullptr, generate_guesses, &threadArgs[t]
            ↪ );

```

```

17     }
18     for (int t = 0; t < num_threads; t++) pthread_join(threads[t], nullptr);
19     total_guesses = guesses.size();

```

下面是对应具体线程函数的实现，以及一个线程参数结构体。

```

1 // 定义线程参数结构体
2 struct ThreadArgs
3 {
4     int start;
5     int end;
6     string base_guess;
7     segment *a;
8     vector<string> *guesses;
9     int init_size;
10 };
11 // 线程函数
12 void *generate_guesses(void *args)
13 {
14     ThreadArgs *threadArgs = (ThreadArgs *)args;
15     for (int i = threadArgs->start; i < threadArgs->end; i++)
16     {
17         string temp = threadArgs->base_guess + threadArgs->a->ordered_values[
            ↪ i];
18         (*threadArgs->guesses)[threadArgs->init_size + i] = temp;
19     }
20     pthread_exit(nullptr);
21 }

```

结果如表5所示，可以看到，在四线程的实现下，是有稍微的加速的，但是还要考虑系统的波动影响。

为啥 pthread 加速效果这么差呢，我们观察代码。在代码中，我们基本上是每 popNext 一个 PT 就创建了四个线程去并行执行任务，虽然这里是平均分的，不需要考虑忙等待的问题。但是，我们 popNext 了很多次，也就是说，我们频繁创建线程，又频繁销毁线程。这里的开销是巨大的，导致加速效果难以提高。

表 5: 结果对比

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
单线程	5.06458	1.00×	0.333787	1.00×	0.300993	1.00×
四线程	4.81589	1.052×	0.305904	1.091×	0.286765	1.050×

2. OpenMP 多线程并行实现

OpenMP 的使用比 pthread 就简单很多了，下面是核心代码部分，我们这里同样用索引访问来解决并发访问的问题。

```

1      int init_size = guesses.size();
2      //这里获取了最多能设置的线程数
3      int max_threads = omp_get_max_threads();
4      guesses.resize(guesses.size() + pt.max_indices[pt.content.size() - 1]);
5      #pragma omp parallel num_threads(max_threads)
6      {
7      #pragma omp for
8          for (int i = 0; i < pt.max_indices[pt.content.size() - 1]; i++)
9          {
10             string temp = guess + a->ordered_values[i];
11             //直接索引写入
12             guesses[init_size + i] = temp;
13         }
14     }
15     total_guesses = guesses.size();

```

实验结果如表6所示。可以看到, OpenMP 的加速效果在开了优化之后是比较明显的。

openmp 虽然也是每次到达这部分代码就调用并行线程,但是它只会在第一次调用的时候创建,然后不使用并行的时候把线程挂起,等到下次使用并行的时候就不需要重新创建线程了,虽然会损失一定空间,但是微乎其微。所以 openmp 能达到好一点的加速效果。

表 6: 结果对比

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
单线程	5.06458	1.00×	0.333787	1.00×	0.300993	1.00×
四线程	4.73815	1.069×	0.260092	1.283×	0.238475	1.262×

3. 加速比与总猜测数

下面,我们来对不同猜测数下的时间进行对比,这里我们把猜测数上限改成一亿,然后在不同优化设置下进行对比。

这里我们一方面增加总猜测数的上限,另一方面只是在多个猜测数节点处记录时间,由于记录时间的开销,这里的总时间会变长,但是不同方式下的开销增加是相同的,而根据糖水不等式,这样只会让加速比减小而不是增大,况且这个开销相当小,可以忽略不计。

不开优化的结果如图2所示,可以发现,在一千万猜测数之内的时候,并行方法的加速比都是在上升趋势的,而猜测数到达一千五百万之后,加速比趋于稳定,分别稳定在 **1.084** (pthread) 和 **1.121** (OpenMP)。

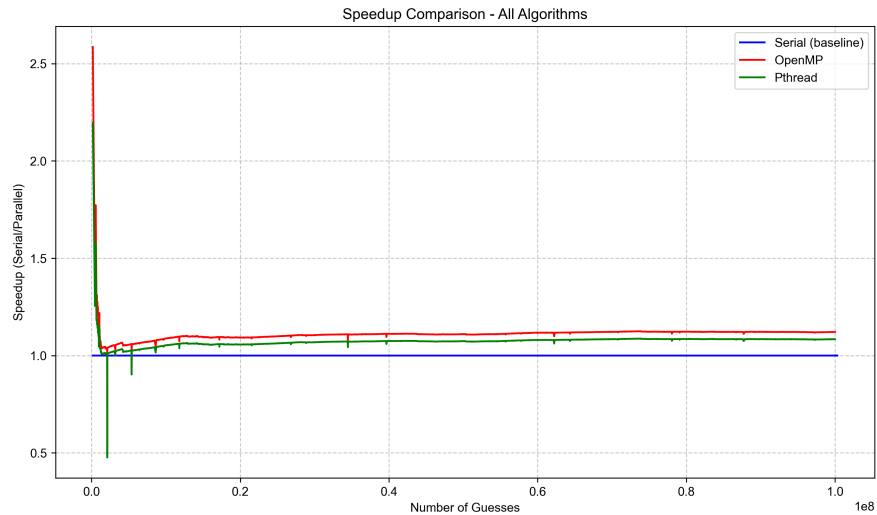


图 2: O0

开启 O1 优化的结果如图3所示, 可以发现开启优化之后, 加速更加明显了, 并且稳定的更加慢, 直到猜测数到达五千万的时候才趋于稳定。此时加速比稳定在 **1.206** (pthread) 和 **1.500** (OpenMP)。

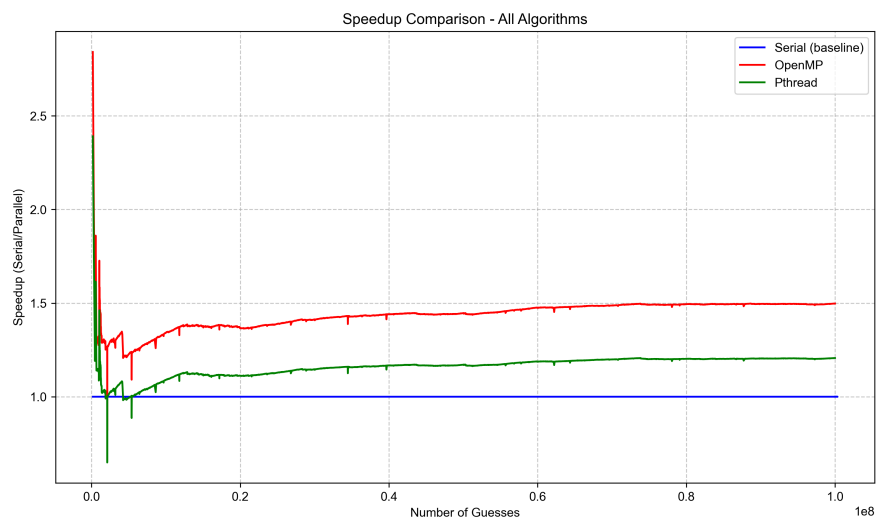


图 3: O1

开启 O2 优化的结果如图4所示, 开启 O2 优化之后, 发现反而加速比还下降了一点, 说明开启 O2 优化之后, 对串行版本的方法优化力度更大, 导致基线变高了。而在 O2 版本下, 趋于稳定的节点更加大了, 甚至在图中还仍在上升趋势。此时加速比在 **1.164** (pthread) 和 **1.460** (OpenMP)。

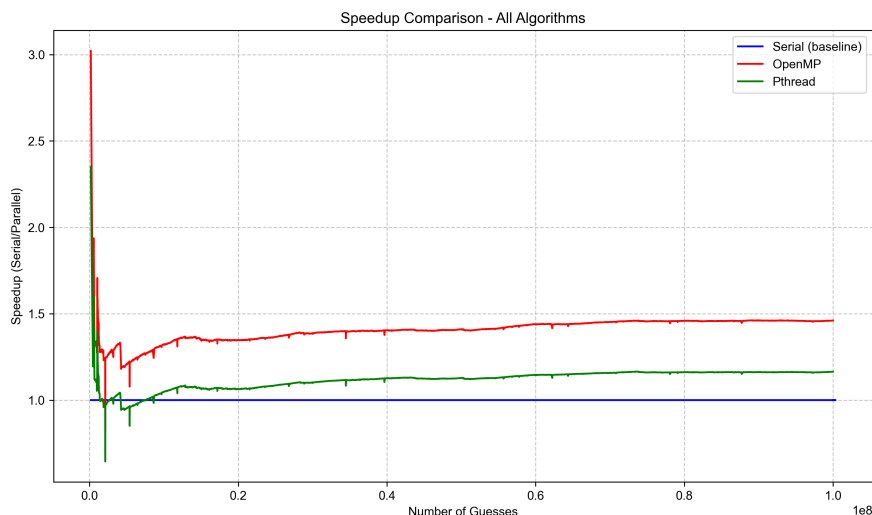


图 4: O2

• 小结与分析

在上面的对比中，我们发现了一个现象：随着猜测数变大，加速比先变大后稳定。

我们知道，并行任务的创建是有开销的。在使用多线程时，每次并行任务启动时有这些额外开销：线程创建/调度的时间、数据划分与任务分发、线程同步与加锁、CPU 缓存一致性维护、false sharing 等低层性能损耗。

而在猜测量比较小的时候，这些开销占大头，分摊到每个猜测计算中的时间比较大，导致加速比较小；当猜测量比较大的时候，这些开销有一部分是固定，这些固定开销被均摊之后特别小，在加速比计算中就可以忽略不计了，所以后面的加速比就趋于稳定了。

所以说，pthread 这个方法是确实实现加速的，但是在猜测量一千万的时候体现并不是很明显。

4. 加速比与并行线程数

我们来探究一下并行线程数对加速比的影响。这里我们在开启 O2 优化的情况下，分别对 pthread 和 OpenMP 两种方法在线程数为 1~8 的情况下进行实验，得到结果如表7所示：（这里我服务器的 cpu 核心为 8，而主线程只有分配任务的使用，所以线程数这里可以开到 8，不存在超出物理核心数的情况。）

表 7: 不同并行线程数结果

线程数	Pthread-O2		OpenMP-O2	
	时间 (s)	加速比	时间 (s)	加速比
1 线程	0.392677	0.77×	0.324796	0.93×
2 线程	0.309910	0.97×	0.260703	1.15×
3 线程	0.283852	1.06×	0.233602	1.29×
4 线程	0.281583	1.07×	0.222322	1.35×
5 线程	0.303205	0.99×	0.216787	1.39×
6 线程	0.311211	0.97×	0.210932	1.43×
7 线程	0.325405	0.92×	0.207917	1.45×
8 线程	0.344234	0.87×	0.218134	1.38×

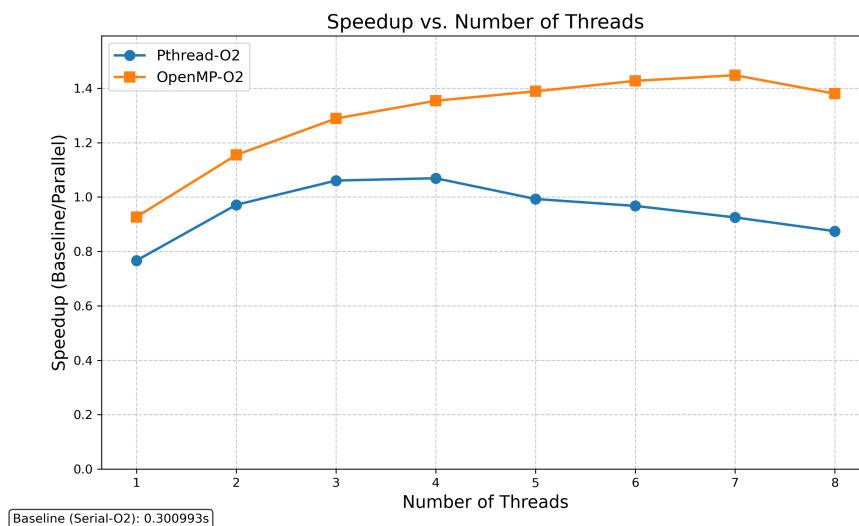


图 5: 不同并行线程数加速比对比

可以看到, 在线程数为 1 的时候, 并行优化都是负优化, 因为存在设置并行的开销。

而当并行线程数增加的时候, 加速比开始上升, 并行带来的加速盖过了并行的额外花销, 实现了并行加速。

但是当并行线程数增加到一定数量的时候, 比如这里 pthread 是 5 之后, openmp 是 8 之后, 加速比开始下降, 发生了并行加速“反转”。是因为:

- 当线程数过大的时候, 分配任务、等待同步的时间反而占了主导, 导致加速比下降。
- 多线程会竞争 L1L2L3 缓存的使用, 结合我这里用的是随机访问, 会导致访问的时候反复大面积跳转, 增加了花销。

可以观察到, pthread 在较小线程数时加速比上升较快, 但很早就到达瓶颈, 其原因主要是:

- 线程管理开销大: pthread 每次都需显式创建、销毁线程, 系统调用成本高, 缺乏线程池机制;
- 任务划分与调度手动完成: 划分不均或任务粒度过细会造成负载不均, 影响性能;

- 缺乏自动优化能力：pthread 不具备 OpenMP 的自动调度、线程复用、核心绑定等机制；

相比之下，OpenMP 使用线程池并具备动态调度能力，因此能在更高线程数下保持较好的加速效果。

(三) 进阶要求

1. SIMD 和多线程同步加速

下面我们把上次实验的 SIMD 四路并行和这次实验的 OpenMP 多线程并行优化结合（这里我们选用效果最好的 7 线程数）。下面是运行结果：

表 8: SIMD 和多线程同步加速结果

优化程度	无优化版本			
	Hash 时间 (s)	加速比	Guess 时间 (s)	加速比
O0	11.7818	1.00×	5.06458	1.00×
O1	6.44946	1.00×	0.333787	1.00×
O2	6.29782	1.00×	0.300993	1.00×

优化程度	SIMD+OpenMP			
	Hash 时间 (s)	加速比	Guess 时间 (s)	加速比
O0	11.3187	1.04×	4.77009	1.06×
O1	4.36641	1.48×	0.230478	1.45×
O2	4.31747	1.46×	0.215457	1.40×

虽然实验指导手册上面说可能不能同时实现优化，但是我们惊喜的发现，在同时优化的情况下，仍然进行了加速（归功于我们选取了最优的多线程数）。

可以看到，不同优化力度下的加速结果是不同的。

在无优化的情况下，加速效果是微乎其微的，但是正如我们前面分析猜测数和加速比的关系所分析的，其实这里如果增加计算量，加速比应该是能够更加大一点，所以这里稍微加速肯定不是系统波动的结果，而是确实确实加速了。

可以看到，O1 优化下的加速比是最大的，无论是 SIMD 还是 OpenMP，而 O2 优化下反而加速比小了一点，因为不同优化下对两个版本代码的优化效果不同。

这里需要解释一下，为什么 Hash 时间对比上一实验差别这么多。因为这里加了一个破解率的检测，但是根据糖水不等式，这里 hash 时间的加速比只会比原来的小而不会变大：

$$\frac{\text{无优化 hash 时间}}{\text{并行优化后 hash 时间}} \geq \frac{\text{无优化 hash 时间} + \text{破解率检测时间}}{\text{并行优化后 hash 时间} + \text{破解率检测时间}}$$

2. 对哈希部分进行 SIMD 和多线程并行优化

两次实验下来，我们可以发现，SIMD 和多线程优化是不同的优化方向，SIMD 是一条代码情况下实现多个计算量同时计算，而多线程是同一时间下实现多条代码的同时计算。而在之前实验中，我们只对 Hash 部分实现了 SIMD，现在，我们尝试在 SIMD 的基础上用 openmp 实现多线程优化（这里同样考虑使用线程数为 7）。

值得注意的是, 由于这个 Md5Hash 部分是为了包饺子, 所以这里原本代码并没有对结果进行存储, 如果加上存储, 这个存储还需要像之前 resize 那个 guesses 一样, 对结果进行随机方法, 来解决并行访问的问题。

由于不需要存储结果, 所以这里只需要考虑对 cracked 这个同时做加法的问题, 我们这里使用原子操作, 来解决并发更改 cracked 的问题。

```

1      alignas(16) bit32 state[4][4];
2      string pw[4];
3
4      // 使用 OpenMP 并行化外层循环
5      #pragma omp parallel for num_threads(7) shared(q, test_set) private(pw,
        ↪ state)
6          for (int i = 0; i < q.guesses.size(); i += 4)
7          {
8              for (int j = i; j < q.guesses.size() && (j - i) < 4; j++)
9              {
10                 pw[j - i] = q.guesses[j];
11                 if (test_set.find(pw[j - i]) != test_set.end())
12                 {
13                     #pragma omp atomic
14                         cracked += 1; //使用原子操作保证线程安全
15                 }
16             }
17             SIMD_MD5Hash(pw, state);
18         }

```

表 9: 结果对比 (基于 Hash time)

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
无优化	11.7818	1.00×	6.44946	1.00×	6.29782	1.00×
SIMD4	11.3559	1.04×	4.40773	1.46×	4.26419	1.48×
SIMD+OpenMP	1.53559	7.67×	0.578018	11.15×	0.593664	10.61×

结果如表所示, 可以看到, 同时在 SIMD 的数据级并行优化以及 OpenMP 的线程级并行优化下, 加速比最好达到了 11.15 倍!!!

由于这里结合了 openmp 和 simd 优化, 比 SIMD 优化快了七倍以上。

下面用 perf 来进行分析对比:

表 10: perf 指标对比

指标	SIMD	SIMD+OpenMP	变化量	变化率
CPU 利用率	1.013	1.140	+0.127	+12.54%
缓存访问次数	211,795,780,779	211,469,692,093	-326,088,686	-0.15%
缓存未命中次数	798,629,688	749,899,449	-48,730,239	-6.10%
缓存未命中率	0.38%	0.35%	-0.03%	-7.89%
分支预测未命中率	1.48%	1.47%	-0.01%	-0.68%
每周期指令数 (IPC)	1.51	1.52	+0.01	+0.66%

可以看到：

- SIMD 的 CPU 利用率为 1.013，而 SIMD+OpenMP 为 1.140，增加了 12.54%。这表明 OpenMP 有效地利用了多核 CPU 资源，提升了并行计算效率。
- SIMD 的缓存未命中率为 0.38%，而 SIMD+OpenMP 为 0.35%，减少了 7.89%，虽然看着减少了一点点，但是这可是所有部分加起来的缓存未命中率，从这个结果可见一斑，Hash 部分的缓存未命中率应该是大大减小了。
- SIMD 的 IPC 为 1.51，而 SIMD+OpenMP 为 1.52，增加了 0.66%。IPC 的小幅提升表明 OpenMP 的并行化提高了指令执行效率。

可以看到，openmp 不仅仅只是对并行部分代码进行了并行化，同时优化了缓存性能，使得加速效果大于了线程数。

3. 多优先队列优化

下面我们尝试对生成猜测以及 PT 优先队列过程进行进一步的 OpenMP 优化。

首先我们观察到，在 init 函数中，遍历了一次 m.ordered_pts，然后把每个 pt 拿出来，初始化之后，计算概率值，然后放进一个 vector 里面，形成一个优先队列，但是这里其实是可以并行化的。我们把这个遍历过程分成几个线程，每个线程存储各自的优先队列，然后最后在合并，核心代码如下：

这里写了一个对多个优先队列归并排序的函数，在对上面所说的遍历过程并行化之后，在归并排序他们的结果。

显然这里的加速过程不会很明显，因为 init 中遍历的 PT 数不会特别多。

Algorithm 1 INITPRIORITY——并行建队与归并**Input:** 训练得到的预排序 PT 列表 *ordered_pts***Output:** 全局优先队列 *priority* (按概率降序)

```

1: function MERGEVECTORS(A, B)                                ▷ 两有序队列归并, 概率降序
2:   if A =  $\emptyset$  then return B
3:   else if B =  $\emptyset$  then return A
4:   else
5:     C  $\leftarrow$  长度  $|A| + |B|$  的数组
6:     STD::MERGE(A, B  $\rightarrow$  C, comp : prob  $\downarrow$ ) return C
7:   end if
8: end function
9: function PARALLELMERGESORT( $\{Q_t\}_{t=0}^{T-1}$ )                    ▷ 多队列并行归并
10:  while  $|\{Q_t\}| > 1$  do
11:    将相邻队列两两配对; 奇数残留则直接下传
12:    parallel for 每对 ( $Q_{2i}, Q_{2i+1}$ )
13:       $R_i \leftarrow$  MERGEVECTORS( $Q_{2i}, Q_{2i+1}$ )
14:    令  $\{Q_t\} \leftarrow \{R_i\}$  继续迭代
15:  end while
16:  return  $\{Q_0\}$  或  $\emptyset$ 
17: end function
18: procedure INITPRIORITY
19:  T  $\leftarrow$  OMP_GET_MAX_THREADS
20:  为每线程置空本地队列 localQ[0..T-1]
21:  parallel for i  $\leftarrow$  0 to  $|\text{ordered\_pts}|-1$ 
22:    pt  $\leftarrow$  ordered_pts[i]
23:    for each seg  $\in$  pt.content do 填充 pt.max_indices (依字母/数字/符号)
24:      pt.preterm_prob  $\leftarrow \frac{\text{preterm\_freq}[pt]}{\text{total\_preterm}}$ 
25:      CALPROB(pt)
26:      将 pt 追加至本线程 localQ[tid]
27:    end parallel for
28:  priority  $\leftarrow$  PARALLELMERGESORT(localQ)
29: end procedure

```

然后我们在来修改 PopNext 函数, 每次只对一个 PT 生成猜测并生成新的猜测 PT 太慢了, 我们希望这里能一次性处理 PT 优先队列中的前几个 PT, 虽然会导致猜测不是最优的, 因为在原本代码中, 下一个要生成猜测的 PT 不一定是第二个, 可能是新插入的 PT。但是这里我们选择牺牲一些颗粒度, 后面结果显示, 这个牺牲是完完全全值得的。

然后每次插入新的 PT 时, 都是从头开始遍历优先队列, 找到合适位置在插进去, 复杂度究极高!

我们考虑一次性处理前四个 PT, 每次处理都并行化的生成猜测, 并行化的生成各自的新 PT 们, 并并行化的对各自的新 PT 们用 sort 进行排序, 得到个各自的优先队列, 然后在用上面设计的归并排序函数, 把这几个优先队列, 以及原本的优先队列进行合并。代码如下:

在设计代码时, 要注意几点:

- 有些地方 openmp 分配任务容易不均匀, 会导致忙等待, 所以在计算各自的优先队列的时

候，我们使用了动态分配任务，防止一些队列比较大，导致计算概率比较久。

- 这里如果嵌套使用 openmp，容易导致资源竞争，而且默认是不允许嵌套使用 openmp 的，这里我们简单点也不嵌套了，所以 Generate 函数里面是原版串行的函数。只是把 guess 改成了本地存储的，而不是全局的（避免并发访问）。

Algorithm 2 POPNEXT4 — 批量弹出与扩展 4 个概率最高的 PT

Input: 全局优先队列 *priority*; 全局猜测表 *guesses*; 计数器 *total_guesses*

Output: 更新后的 *priority*, *guesses*, *total_guesses*

```

1:  $k \leftarrow 4$ ; 为  $k$  个线程各准备局部容器 new_pts[], local_guesses[], local_num[]
2: parallel for  $t \leftarrow 0$  to  $k-1$  do
3:   critical  $pt \leftarrow \text{POPFRONT}(priority)$ 
4:   if  $pt \neq \emptyset$  then
5:      $(g, n) \leftarrow \text{GENERATE}(pt)$  ▷ 生成局部猜测串
6:      $local\_guesses[t] \leftarrow g$ ;  $local\_num[t] \leftarrow n$ 
7:      $new\_pts[t] \leftarrow pt.\text{NEWPTS}()$ 
8:   end parallel for
9:  $total\_guesses \leftarrow total\_guesses + \sum_{t=0}^{k-1} local\_num[t]$ 
10:  $guesses.\text{RESIZE}(total\_guesses)$  并按顺序追加所有 local_guesses
11: parallel for  $t \leftarrow 0$  to  $k-1$  do
12:   for each  $q \in new\_pts[t]$  do  $\text{CALPROB}(q)$ 
13:    $\text{SORT}(new\_pts[t], \text{概率降序})$ 
14: end parallel for
15:  $all\_new \leftarrow \text{PARALLELMERGESORT}(new\_pts)$ 
16:  $priority \leftarrow \text{MERGEVECTORS}(priority, all\_new)$ 

```

表 11: 超级并行优化结果对比

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
无优化	5.06458	1.00×	0.333787	1.00×	0.300993	1.00×
openmp 优化	4.73815	1.07×	0.260092	1.28×	0.238475	1.26×
超级 OpenMP 优化	0.5894	8.59×	0.170608	1.96×	0.168509	1.79×

结果如表11所示，超级夸张，在记录数据的时候一度怀疑是不是写错了，但是反复确认了一下，同一优化下 Train time 是差不多的。

在超级 OpenMP 优化中，破解个数仅仅下降到了 **291267**，减少了 **18.7%**，但是加速比是相当可以的。甚至在不开优化的时候，加速比到达了 **8.59x**，也应证了我前面的观察，这里估计是因为反复对 PT 插入排序导致时间花销巨大。

但是在开启优化之后，这个加速比的差距被大大降低，可以合理猜测，这里反复的插入排序被优化掉了。

4. 并行优化思考

1. 并行化适合处理什么样的问题？

并行实际上是对多余资源的充分利用，但是同时会产生线程调度开销、同步与通信的成本、负载不均、资源争夺的问题。所以并不是所有任务都适合并行的，一旦这些问题产生的额外开销盖过了并行带来的优化，那么这个并行又耗费时间又耗费计算资源。

所以，并行化适合处理一些计算密集型任务、可分解问题型、大批量数据的任务。这些任务往往可以分解成诸多个子任务，并且任务之间的相互依赖关系比较弱。对这些问题并行化，能够充分利用计算资源的同时加快程序运行。

2. 什么时候并行化更合适？什么时候串行更合适？

并行和串行往往不是非黑即白的，而是相互配合。

当任务存在较高的相互依赖性的时候，此时同步和通信带来的时间花销容易超过并行带来的优化，这时候就应该选择使用串行化。

而当任务中存在一部分代码，他们的前后依赖性比较低的时候，此时并行化是一个很好的选择。

当然，不是一遇到无依赖可分解的任务就直接并行化，分解的子任务最好比较大，否则，频繁的调用并行线程，线程调度的开销同样会让并行化得不偿失，反而比串行更差。

还需要考虑的是计算资源的大小，如果分解后的并行子任务对计算资源的要求还是很大，那么并行分配给这个子任务的计算资源可能不够，可能导致资源争夺的情况发生。（比如，每个任务只有一个 cpu，那么每个任务的缓冲区就会很小，导致缓存命中率低，可能导致执行效率下降）。

四、 口令猜测算法的 MPI 多进程优化

（一） MPI 并行化原理与适用性分析

MPI (Message Passing Interface) 是一种广泛应用于分布式内存并行计算的通信标准，适用于多节点、多进程协同处理大规模计算任务。在口令猜测算法中，候选口令的生成与验证过程天然具备高度的数据独立性，各进程间无需共享中间状态，极适合采用 MPI 进行任务划分与负载均衡。

与 SIMD、OpenMP 等共享内存并行方式相比，MPI 能够突破单机内存和核心数的限制，支持在多台主机间扩展计算规模。其核心思想是将任务空间划分为若干子区间，每个进程独立负责一部分候选口令的生成与哈希验证，通过消息传递机制实现最终结果的收集与合并。

（二） MPI 并行化设计与实现思路

1. 任务划分与进程协作

在本实验中，采用静态任务划分策略。假设总共有 N 个候选口令需要处理， P 个 MPI 进程，则每个进程负责约 $\lceil N/P \rceil$ 个口令的处理。主进程 (rank 0) 负责任务分发与最终结果收集，其余工作进程 (rank > 0) 独立执行分配到的子任务，并将结果通过 MPI 发送回主进程。

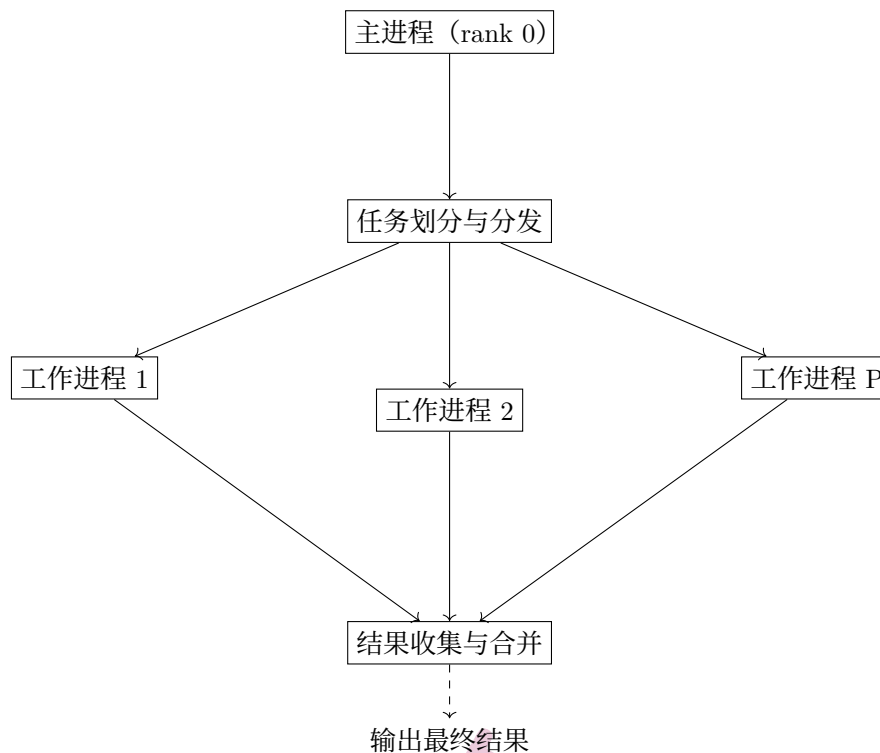


图 6: 基于 MPI 的口令猜测并行流程

2. 核心实现代码

MPI 并行化的核心代码如下所示:

```

1  #include <mpi.h>
2  #define N 1000000 // 假设总任务数
3  int main(int argc, char *argv[]) {
4      int rank, size;
5      MPI_Init(&argc, &argv);
6      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
7      MPI_Comm_size(MPI_COMM_WORLD, &size);
8
9      int chunk = (N + size - 1) / size;
10     int start = rank * chunk;
11     int end = (start + chunk > N) ? N : (start + chunk);
12
13     // 每个进程独立处理自己的任务区间
14     for (int i = start; i < end; ++i) {
15         // 生成第i个候选口令并进行拼接
16         // ...
17     }
18
19     // 结果收集 (如找到目标口令则通知主进程)
20     // MPI_Gather/MPI_Reduce等
21     MPI_Finalize();
22     return 0;
  
```

23 }

(三) MPI 并行化流水线设计

可以观察到，生成口令和口令哈希是互相独立的，但是在原算法中，生成一批口令才会对其哈希，这里可以利用多线程的特性，一个线程来生成口令，生成口令的同时，可以让另一个线程进行口令的哈希，这样两个部分可以同时进行，使得总时间变小，如图7所示。

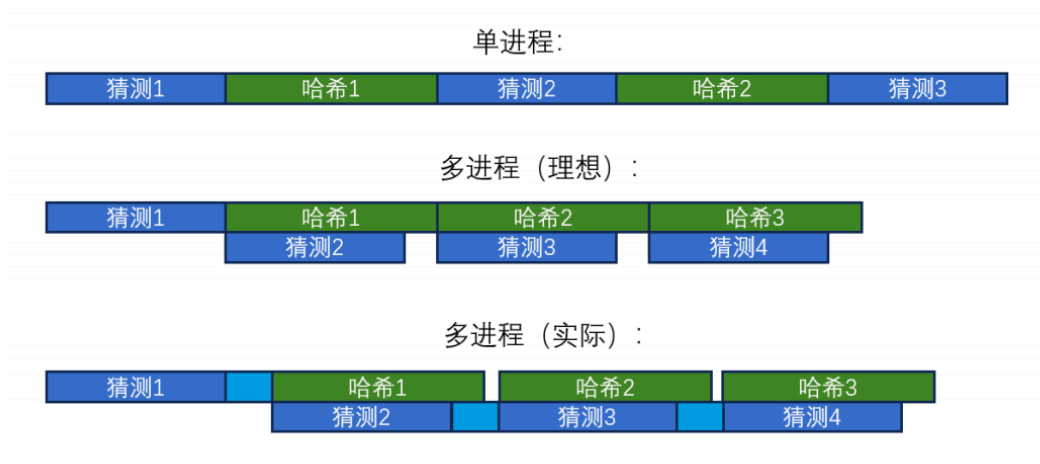


图 7: 流水线示例

1. MPI 并行化流水线思路

只需要两个进程，分别用来做生成猜测和哈希结果，思路如下：

1. 先是两个进程同时进入 while 循环，然后先是 rank0 进程进行猜测生成。
2. 当猜测数量达到上限时，发送开始信号给 rank1 进程，然后这两个进程开始阻塞通信，rank0 进程发送辅助信息以及猜测字符串给 rank1 进程。
3. 当所有通信被接受之后，此时 rank0 进程继续生成猜测，rank1 进程开始哈希。
4. 如果 rank1 先完成了哈希，那么会阻塞在接收开始信号，等待 rank0 的猜测结果。如果 rank0 先完成，那么会等待 rank1 的接收下一次的开始信号。
5. 重复上面 2~4 过程，直到 rank0 不在生成猜测，此时 rank0 发送结束信号给 rank1，两个进程都 break 退出 while，结束 MPI 进程调用。

2. MPI 并行化流水线核心代码

流程说明 算法 3 将候选口令生成 (Guess) 与哈希验证 (Hash) 解耦并映射到 rank 0 与 rank 1 两个进程，实现经典“生产者-消费者”流水线。整体执行过程如下：

1. **初始化阶段** 进程启动后首先调用 MPI_Wtime 记录起始时间 t_0 并获得自身 rank。只有 rank 0 与 rank 1 参与计算，其余进程立刻 MPI_Finalize 退出，避免资源浪费与通信干扰。

2. Guess 生产线程 (rank = 0)

- 维护循环计数器 `curr_num` 与历史生成口令数 `history`。
- 每生成一条候选口令即压入本地缓存。当 `curr_num > TH` 时触发**批量发送**:
 - (a) 并行计算每条口令长度 `lengths[i]` 及总长度 `total_len`;
 - (b) 构造连续缓冲区 `all_guesses`, 并生成偏移量 `offsets[i]`;
 - (c) 先发送“**开始哈希信号**”, 随后一次性发送 $\langle num, total_len, lengths, all_guesses \rangle$; `history`。
- 当 `history` 达到搜索上限时发送“**结束信号**”, 记录流水线总耗时 t_{pipe} 并跳出循环。

3. Hash 消费线程 (rank = 1)

- 反复阻塞等待 rank 0 发送的 `signal`
- 借助 OpenMP 将 `num` 条口令均匀划分到各 CPU 核心, 通过偏移量 `offsets` 快速切片子串并执行哈希。每发现一次碰撞即本地累加 `local_hit`, 最后使用 `atomic` 指令归约到 `global_hit`, 避免锁竞争。

(四) 实验结果与分析

为了便于对比, 本节将**基础 MPI 并行**、**通信时间解析**与 *Guess-Hash 流水线*三组实验结果进行统一汇总, 并围绕性能走势给出原因分析与改进思路。

1. 基础 MPI 实现性能对比

表 12 汇总了原版、OpenMP 优化版与基础 MPI 优化版在不同编译等级下的口令“猜测”阶段耗时(秒)及相对加速比。可以看到:

- OpenMP 在 -O1/-O2 均表现出稳定的 $1.3\times$ 左右加速;
- MPI 方案因 5-6 次阻塞通信掩盖了并行收益, 最高仅达 $0.57\times$ (o0), 在 -O2 甚至跌至 $0.08\times$ 。

表 12: 不同版本在不同优化等级下的“Guess”阶段耗时与加速比

优化等级	原版	OpenMP	MPI	
	Time (s)	Speedup	Time (s)	Speedup
o0	7.81	$1.04\times$	13.81	$0.57\times$
o1	0.61	$1.27\times$	4.34	$0.14\times$
o2	0.63	$1.36\times$	7.46	$0.08\times$

2. 通信时间与阻塞开销解析

表 ?? 给出了三种编译等级下的通信耗时及其占总时间比例。可见 -O2 默认为 `std::copy→memcpy` 展开, 导致缓存失配与写缓冲同步, 通信耗时激增至 3.76 s, 占比 44%。

Algorithm 3 MPI 流水线并行 Guess + Hash**Input:** 搜索上限 MAX , 批量阈值 TH , 仅使用 $rank 0, 1$ **Output:** 找到的哈希碰撞次数 $global_hit$

```

1: 初始化 MPI, 获得  $rank$ , 记录起始时间  $t_0 \leftarrow MPI\_Wtime()$ 
2: if  $rank > 1$  then
3:   MPI_Finalize(); return
4: end if
5: if  $rank = 0$  then ▷ Guess
6:    $history \leftarrow 0, curr\_num \leftarrow 0, q.guesses \leftarrow \emptyset$ 
7:   while true do
8:     生成新候选口令  $pwd$ , 压入  $q.guesses$ ,  $curr\_num++$ 
9:     if  $curr\_num > TH$  then ▷ 达到阈值, 封装并批量发送
10:      计算  $num \leftarrow |q.guesses|$ 
11:      并行求每串长度  $lengths[i]$ , 并累加得到  $total\_len$ 
12:      计算偏移量数组  $offsets[i]$ 
13:      将所有口令拷贝到连续缓冲区  $all\_guesses$ 
14:      MPI_Send( $start\_signal = 1$ )
15:      依次 MPI_Send( $num, total\_len, lengths, all\_guesses$ )
16:       $history \leftarrow history + curr\_num; curr\_num \leftarrow 0; q.guesses \leftarrow \emptyset$ 
17:    end if
18:    if  $history \geq MAX$  then ▷ 达到总搜索上限
19:      MPI_Send( $end\_signal = -1$ )
20:      记录流水线总耗时  $t_{pipe} \leftarrow MPI\_Wtime() - t_0$ 
21:      break
22:    end if
23:  end while
24: else ▷ Hash, rank=1
25:    $global\_hit \leftarrow 0$ 
26:   while true do
27:    MPI_Recv( $signal$ )
28:    if  $signal = -1$  then ▷ 接收到结束信号
29:      break
30:    end if
31:    MPI_Recv( $num, total\_len, lengths, all\_guesses$ )
32:    使用 OpenMP 并行:
33:    for  $i \leftarrow 0$  to  $num - 1$  do
34:      取子串  $s_i \leftarrow all\_guesses[offsets[i] : lengths[i]]$ 
35:      if  $validate\_hash(s_i)$  then  $local\_hit++$ 
36:    end for
37:    原子加:  $global\_hitlocal\_hit$ 
38:  end while
39: end if
40: MPI_Finalize(), 输出  $global\_hit, t_{pipe}$ 

```

优化等级	通信时间 (s)	通信占比 (%)
o0	2.16	11.96
o1	0.67	12.05
o2	3.76	43.99
o2_v2 [†]	0.19	3.79

表 13: “Guess” 阶段通信耗时及占比 (基础 MPI)

注: o2_v2 指禁用 memcpy 优化后的版本

结论 通信与计算失衡是基础 MPI 方案失速主因;编译器的“有害”优化应通过 `-fno-builtin-memcpy` 等选项加以规避。

3. Guess-Hash 流水线性能

将“生产者-消费者”思想引入两进程流水线后, Guess 与 Hash 可以时间重叠。表 ?? 展示了整体耗时与加速比:

优化等级	原版 Guess+Hash (s)	Pipeline (s)	Speedup
o0	17.43	14.14	0.81×
o1	3.73	7.10	0.53×
o2	3.76	7.48	0.50×
o2_v2	3.76	3.04	1.24×

表 14: Guess+Hash 流水线性能对比

- 在默认 `-O2` 下流水线并未受益;
- 关闭 `memcpy` 展开 (`o2_v2`) 后通信延迟大幅降低, 整体实现 24% 加速。

4. 综合分析与改进方向

- 1) **通信开销占主导**: 高频阻塞 + 大块 `memcpy` 导致缓存争用, 使理论并行收益被吞噬。
- 2) **流水线潜力可观**: 当通信延迟被压缩至 $< 5\%$, Guess 与 Hash 可近乎满幅重叠。
- 3) **建议优化**:
 - 使用 `MPI_Isend/Irecv` 与双缓冲削弱同步点;
 - 动态批量阈值自适应平衡“气泡”;
 - 在编译层显式关闭对通信不友好的内存内联/展开, 或采用更细粒度 SIMD 拷贝。

综上, **通信与计算协同**是决定本任务并行效率的核心因素;合理的流水线设计与“友好”编译优化相结合, 可为后续 GPU/MPI 混合或多级流水线扩展奠定坚实基础。

五、 口令猜测算法的 GPU 加速优化

(一) 关于口令猜测算法 GPU 加速优化的思考

1. GPU 并行优化原理

GPU（图形处理单元）以其高度并行的计算能力，成为加速大规模数据处理和计算密集型任务的重要工具。不同于 CPU 的少量复杂核心，GPU 包含大量简单计算单元，适合处理具有数据独立性和重复性强的任务。CUDA 是 NVIDIA 推出的并行计算架构，允许开发者以 C/C++ 语言编写并运行 GPU 内核函数，通过线程块与线程网格的组织方式进行大规模并行任务调度。

GPU 优化通常包含以下几个关键方面：

- **任务并行化设计**：识别可独立并行的任务单元，合理分配线程。
- **内存访问优化**：优化全局内存读写、利用共享内存、避免非对齐访问。
- **数据传输控制**：减少 Host-Device 之间的数据交换频率和数据量。
- **资源调度策略**：合理配置线程数与块大小，提高计算资源利用率。

2. GPU 并行加速对口令猜测算法的适用性思考

口令猜测算法通常由两个核心步骤组成：候选口令生成（如字符串拼接）与概率计算。在对 GPU 加速适用性进行评估时，需重点考察任务的计算密集度与可并行性。

在基础任务中，字符串拼接属于轻计算、重传输的操作，涉及 STL 容器、复杂结构转换与频繁内存分配，GPU 优化带来的收益难以抵消数据传输的开销。在此情形下，GPU 的并行优势无法有效发挥。

相比之下，概率计算模块涉及大量重复的乘除操作，结构规整、数据量大且彼此独立，具备典型的并行计算特性。将该模块交由 GPU 批量处理，既能减少主机重复计算压力，也能提升整体吞吐量，因此成为更具优化潜力的 GPU 加速目标。

(二) GPU 并行加速优化算法的设计

1. 基础 GPU 并行

在基础任务中，实验目标为将字符串拼接逻辑迁移至 GPU 进行并行处理。由于原始实现中采用了 C++ STL 结构，而 GPU 仅支持 C 风格数组，因此首先需将输入字符串数据扁平化为连续内存结构，并构建相应的偏移量辅助数组以完成定位。

CUDA 核函数负责在每个线程中拼接前缀与对应目标字符串，最终生成完整候选口令。核函数逻辑如下：

```
1 __global__ void processStringKernel(  
2     const char* prefix, int prefix_len,  
3     const char* values_data, int* lengths, int* offsets,  
4     int num, char* output, int* output_offsets) {  
5     int idx = blockIdx.x * blockDim.x + threadIdx.x;  
6     if (idx < num) {  
7         int val_len = lengths[idx];  
8         int val_offset = offsets[idx];  
9         int result_len = prefix_len + val_len;  
10        int out_offset = output_offsets[idx];
```

```

11
12     for (int i = 0; i < prefix_len; i++) {
13         output[out_offset + i] = prefix[i];
14     }
15     for (int i = 0; i < val_len; i++) {
16         output[out_offset + prefix_len + i] = values_data[val_offset + i
17             ↪ ];
18     }
19     output[out_offset + result_len] = '\0';
20 }

```

2. GPU 多 PT 并行优化-GPU 概率计算加速

为进一步发挥 GPU 在处理重复性浮点计算方面的优势，实验中将新 PT 集合的概率计算并行化。每个 PT 的概率通过多个乘数与除数计算得出，具备高度独立性和一致性。

为此，将每个 PT 的乘数、除数及初始概率整理打包，并统一传输至 GPU 设备端。每个线程负责一个 PT 概率的完整计算流程：

```

1 __global__ void calculateProbabilityKernel(
2     float* preterm_prob, float* multiplier, float* divisor,
3     int* offsets, float* results, int num_pts) {
4     int pt_idx = blockIdx.x * blockDim.x + threadIdx.x;
5     if (pt_idx < num_pts) {
6         int start = offsets[pt_idx];
7         int end = offsets[pt_idx + 1];
8         float probability = preterm_prob[pt_idx];
9         for (int i = start; i < end; i++) {
10             if (divisor[i] > 0)
11                 probability *= (multiplier[i] / divisor[i]);
12         }
13         results[pt_idx] = probability;
14     }
15 }

```

3. 基础 GPU 代码实现

基本 GPU 拼接任务的完整流程包括数据打包、设备内存分配与复制、核函数调用与结果还原。通过实验对比发现，在优化级别 O1 与 O2 下，GPU 版本的猜测时间分别为 2.27 秒与 2.23 秒，相较于 CPU 的 0.43 秒并无提升。nvprof 分析显示，主机-设备数据拷贝占总时间的 90% 以上，内核计算仅占 9.61%。GPU 并未充分利用其计算优势，主要受限于数据传输瓶颈。

4. GPU 并行优化代码实现

概率计算 GPU 化实现中，初期步骤在 CPU 侧完成模型查表、数据打包，随后将批量乘除因子传入 GPU 执行核函数。在函数实现过程中，遇到 `std::bad_alloc` 内存分配失败异常，表明单次分配的数组体积可能过大。

实验尝试分批处理以降低峰值内存压力，尽管将每批 PT 控制在 1000 以内，但仍存在崩溃现象，说明需进一步优化内存布局与分配策略。

(三) 实验结果分析

1. 不同优化级别下字符串拼接任务的加速比评估

在不同 CPU 优化级别 (O0/O1/O2) 下，比较 GPU 并行与 CPU 串行实现的性能，实验结果如表 15 所示。

表 15: GPU 加速与原版 GuessTime 性能对比 (时间单位: 秒)

优化级别	原版时间 (s)	GPU 时间 (s)	加速比
O0	7.53	8.87	0.85
O1	0.43	2.27	0.19
O2	0.43	2.23	0.19

实验结果表明，GPU 加速方案在不同优化级别下均未展现出预期的性能优势。在未开启编译器优化时，GPU 实现耗时 8.87 秒，相比 CPU 串行版本的 7.53 秒，实际获得了 0.85 倍的相对性能，这意味着 GPU 实现反而比 CPU 版本慢了约 15%。这一现象与常见的 GPU 加速预期形成鲜明对比，值得深入分析。

当开启 O1 和 O2 优化后，CPU 串行版本的性能出现显著提升，执行时间从 7.53 秒骤降至 0.43 秒，优化效果极为明显。然而 GPU 版本在相同优化级别下的性能改善有限，O1 和 O2 级别下分别耗时 2.27 秒和 2.23 秒，导致加速比进一步降低至 0.19 倍。这种差异揭示了两个关键问题：一方面，现代编译器对 CPU 代码的优化能力已十分强大，特别是对字符串处理等常规计算任务；另一方面，当前的 GPU 实现可能存在数据传输瓶颈或计算资源利用率不足的问题。

进一步分析可以发现，字符串拼接任务本身的特点可能是导致 GPU 优势无法发挥的主要原因。这类任务通常具有较高的分支预测复杂度，且每个操作的计算强度较低，难以充分利用 GPU 的大规模并行计算能力。同时，频繁的数据传输开销可能抵消了并行计算带来的收益，特别是在处理规模相对较小（秒级）的任务时，GPU 的固定启动开销变得尤为明显。这些因素共同作用，解释了为何在各级优化下 GPU 实现都未能超越 CPU 版本。

2. GPU 活动时间分布分析

使用 nvprof 工具分析 GPU 各部分运行时分布，结果见图 8。

直观展示了不同优化级别下 GPU 活动时间分布的对比，进一步印证了数据传输（尤其是 HtoD 拷贝）在整体性能中的主导作用。优化级别对内核执行时间的改善效果有限，说明当前 GPU 并行实现受限于数据传输效率，而非计算能力。未来优化方向可包括减少数据传输频率、优化内存分配策略或采用异步传输技术，以降低数据拷贝开销，提升整体性能。

3. 实验小结

实验表明，GPU 并非对所有任务均适用。基础任务中的字符串拼接受限于传输与结构打包开销，优化效果不佳。而进阶任务中的概率计算展现出 GPU 并行的潜力，但仍面临内存分配与调试难题。

为进一步优化 GPU 加速效果，建议：

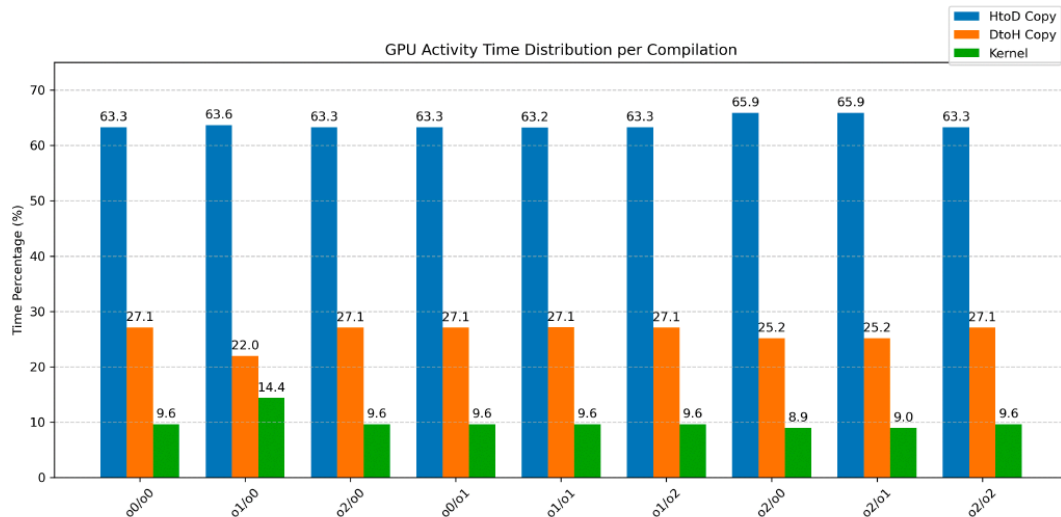


图 8: GPU 活动时间对比

- 优化数据布局，减少数据打包复杂度。
- 利用 CUDA 流进行异步传输，减少同步等待。
- 分批处理与资源池设计，避免内存峰值溢出。
- 探索 CPU-GPU 协同任务划分，提升系统整体效率。

六、 口令猜测算法新探索

(一) 密码切割并行化

在一个学期的尝试中，我们总是对 guess 部分和 hash 部分进行加速尝试，但是口令猜测算法的大头一直是 train 部分，回看代码，对于每个原密码，先是拆分片段，然后对片段的概率结果进行统计，虽然概率结果这里有强依赖性，但是片段拆分是互相独立的。

结合前面的思考和对之前多种并行方法对分析，在这里，并不适合用 SIMD 和 GPU 进行加速，因为有很多条件分支判断，这里只适合用多线程或者多进程，当然，在 cpu 资源富余的情况下，还是尽量使用多线程，因为多进程会带来通信开销。再加上我是在 GPU 服务器上面进行实验尝试，下面，对片段拆分这个步骤进行多线程并行化：

1. parse 原函数整体流程及代码

- **输入：**单个密码字符串 pw。
- **分割：**按字符类型（字母、数字、符号）对密码进行分段。
- **统计：**每遇到一个新片段，查找或新建对应的 segment（按 type+length），并将当前内容插入 segment 的 value 集合，同时更新频率。
- **PT 构建：**将每个 segment 按顺序插入 PT 对象。
- **PT 统计：**查找或新建 PT，更新 PT 频率。

主要代码流程伪代码:

```

1  for (char ch : pw) {
2      if (isalpha(ch)) {
3          // 处理字母段
4          if (curr_type != 1) {
5              // 处理前一个段
6              segment seg(curr_type, curr_part.length());
7              if (FindXXX(seg) == -1) {
8                  新建segment并插入全局集合
9              } else {
10                 更新segment频率
11             }
12             curr_part.clear();
13             pt.insert(seg);
14         }
15         curr_type = 1;
16         curr_part += ch;
17     }
18     // ... 处理数字和符号同理 ...
19 }
20 // 处理最后一个段
21 if (!curr_part.empty()) {
22     segment seg(curr_type, curr_part.length());
23     // 查找或新建segment, 插入value, 更新freq
24     pt.insert(seg);
25 }
26 // 查找或新建PT, 更新freq

```

2. parse_much 并行实现思路分析及代码实现

观察原本的函数, 可以发现, 它是对每个 pw 进行分割之后, 每分割出一个片段, 就加入总统计里面并进行概率统计。在概率统计这部分, 是有数据依赖和变量共用的。但是在分割这部分, 是可以独立进行的, 但是要注意的是, 这里记录片段的时候需要记录片段的生成顺序, 否则结果将会不同!

整体思路:

- **输入:** 一批密码字符串。
- **分割阶段 (并行):** 使用 OpenMP 等多线程技术, 对每个密码独立分割, 得到每个密码的分段结果。
- **汇总阶段 (串行):** 依次遍历每个密码的分割结果, 严格模拟 parse 的统计逻辑, 将每个 segment 和 PT 插入全局数据结构并更新频率。

```

1  // 并行分割
2  #pragma omp parallel for
3  for (int p = 0; p < pw_num; ++p) {
4      vector<pair<int, string>> segs;

```

```

5 // 对pws[p]分割, 得到segs
6 segs_type_value[p] = segs;
7 }
8
9 // 串行汇总
10 for (int p = 0; p < pw_num; ++p) {
11     PT pt;
12     for (auto seginfo : segs_type_value[p]) {
13         segment seg(seginfo.type, seginfo.value.length());
14         if (FindXXX(seg) == -1) {
15             新建segment并插入全局集合
16         } else {
17             更新segment频率
18         }
19         segment.insert(seginfo.value);
20         pt.insert(seg);
21     }
22     // 查找或新建PT, 更新freq
23 }

```

3. 特点与对比

- 分割阶段并行, 提升了处理速度。
- 汇总阶段依然串行, 保证统计逻辑与原 parse 一致。
- 由于 segment 唯一性判定依然只用 type+length, value 集合和 freq 的合并需特别注意, 防止统计丢失。
- 批量处理时全局数据结构的插入顺序需要能与串行 parse 相同, 所以这里我们需要记录顺序, 统计内容也应保持一致。

(二) 概率计算并行化

在之前的 GPU 实验中, 对概率计算的并行化并不成功, 这里重新对这个方法进行尝试:

思路还是和原本一样, 在 popNext 部分, 对生成的所有新 PT 统一进行概率计算, 充分利用 GPU 快速计算的能力:

- **批量处理:** 将待计算的 PT 分批, 每批次处理一定数量的 PT, 避免一次性分配过大内存。
- **数据展开:** 对每个 PT, 预先展开其概率初值、乘数 (multiplier)、除数 (divisor) 等数据, 组织为连续数组, 便于 GPU 访问。
- **CUDA 核函数:** 每个线程负责计算一个 PT 的最终概率, 遍历其所有乘数和除数, 完成累乘累除。
- **结果回传:** 计算结果从 GPU 拷贝回主机内存, 写回原 PT 对象。
- **异常处理:** 若 GPU 内存不足或 CUDA 调用失败, 自动切换为 CPU 串行/批量计算, 保证健壮性。

1. 代码实现

主机端批量处理流程

```

1  for each batch in all_batches:
2      // 1. 数据准备
3      for each PT in batch:
4          preterm_prob[i] = PT.prob
5          offsets[i] = total_count
6          for each idx in PT.curr_indices:
7              multiplier[k] = ...
8              divisor[k] = ...
9              k++
10         offsets[batch_size] = total_count
11         // 2. 拷贝数据到GPU
12         cudaMemcpy(...)
13         // 3. 启动CUDA核函数
14         calculateProbabilityKernel<<<numBlocks, blockSize>>>(<
15             preterm_prob, multiplier, divisor, offsets, results, batch_size
16         )
17         // 4. 拷贝结果回主机
18         cudaMemcpy(...)
19         // 5. 写回PT对象
20         for i in batch_size:
21             PT[i].prob = results[i]

```

CUDA 核函数:

```

1  __global__ void calculateProbabilityKernel(...){
2      int pt_idx = blockIdx.x * blockDim.x + threadIdx.x;
3      if (pt_idx < num_pts) {
4          int start = offsets[pt_idx];
5          int end = offsets[pt_idx + 1];
6          float probability = preterm_prob[pt_idx];
7          for (int i = start; i < end; i++) {
8              if (divisor[i] > 0)
9                  probability *= (multiplier[i] / divisor[i]);
10         }
11         results[pt_idx] = probability;
12     }
13 }

```

(三) 实验结果及分析

下面我们将所有优化进行集合然后得到运行结果，优化分别为：Hash 部分的 SIMD 加速和 openmp 多线程加速、train 部分的口令切割并行、Guess 部分的概率 GPU 并行优化和 openmp 优化、多 PT 的 openmp 优化。

发现加入上面两个优化之后结果反而不是很好，尽管我分开方法运行结果，都无法比原本优化基础上实现加速。

表 16: 密码破解性能优化对比 (时间单位: 秒)

优化级别	原版代码			优化代码			加速比	
	Guess	Hash	Train	Guess	Hash	Train	Guess	Hash
o0	7.5328	6.17275	62.9035	1.94494	0.98426	61.2217	3.87×	6.27×
o1	0.431917	2.24207	12.8671	0.679047	0.271932	12.7961	0.64×	8.24×
o2	0.433922	2.13889	12.1954	0.437461	0.215413	12.7107	0.99×	9.93×

一方面,发现每次 GPU 的概率计算大多只有几个,每次生成的 PT 并不是很多导致其性能瓶颈:首先是数据传输开销,每次启动 GPU 核函数都需要在主机与设备间传输 PT 数据,当计算量较小时通信时间占比过高;其次是并行度不足问题,实际生成的 PT 数量往往难以填满 GPU 的计算单元;最后是计算负载不均衡,不同 PT 的计算复杂度差异导致线程束效率下降。这些因素共同解释了为何在 o1 级别会出现 0.64 倍的负加速现象。类似地,口令切割并行化的效果受限也源于多重因素:全局数据结构的锁竞争增加了同步等待时间,中间结果的存储消耗了额外内存带宽,而细粒度并行带来的任务调度开销在小型数据集上尤为明显

另一方面,虽然密码字符串的切割过程本身可以并行执行,但后续的全局统计操作必须串行进行,这形成了典型的 Amdahl 定律约束。特别是在处理较小规模的数据集时(如 o1/o2 级别),并行任务调度和同步的开销完全抵消了理论上的加速收益。

七、 总结

本报告围绕口令猜测算法的多层次并行优化展开,以 PCFG 生成模型为起点,系统比较了 SIMD、pthread、OpenMP、GPU 以及 MPI 等一系列并行技术在 *Guess*、*Hash* 与 *Train* 三大阶段中的性能得失。实验全部在 00/01/02 编译等级下复现,确保结论具备可重复性和横向可比性。

数据层并行: SIMD 的边界

针对 MD5 哈希,四路 SIMD (ARM NEON / x86 SSE) 可在 o2 获得 $1.95\text{--}2.16\times$ 加速;但对字符串拼接、概率队列维护等多分支逻辑则收效甚微。瓶颈源于数据重排、循环移位延迟与分支预测失效,显示 SIMD 更适合纯粹算术密集而非条件纷繁的任务片段。

线程层并行: pthread 与 OpenMP 的取舍

pthread 需频繁 *create/join*, 管理开销抵消收益;四线程版本最高仅 $1.09\times$ 。OpenMP 维持常驻线程池并自动负载均衡,在 7 线程时可达 $1.45\times$,并与 SIMD 形成“指令级+线程级”双层并行,o1 综合加速比提升至 $1.48\times$ 。进一步通过多 PT 批处理与本地队列归并,OpenMP 在 *Guess* 阶段取得 $1.96\times$,为 CPU 侧最具性价比的方案。

进程/节点层并行: MPI 与流水线

基础 MPI 静态划分在单机上反而拖慢运行,通信与缓存失配导致 o2 加速比仅 $0.08\times$ 。引入“两进程 *Guess-Hash* 流水线”并关闭有害 *memcpy* 展开后,整体可获 24% 提升,说明「通信隐藏+编译策略」是分布式加速的先决条件。

GPU 加速：潜力与桎梏

GPU 对轻量级字符串拼接表现不佳，90% 以上时间耗于 Host-Device 拷贝；哈希概率批量计算虽具并行潜力，但单批规模受显存与 `std::bad_alloc` 限制，整体加速未及预期。未来需依赖异步流、分块调度及混合 CPU/GPU 协同，方能释放 GPU 计算吞吐。

训练阶段的新尝试

本次实验首次将「密码切割」从单条流水转化为**两阶段**流程：先用 OpenMP 对每条密码的分段操作并行化，随后在主线程顺序合并 segment 与 PT 的全局频次。该策略理论上可将分割时延按线程数近似等分，但实践表明整体 Train 时长仅缩短数个百分点。根因在于：

- (a) **全局唯一性约束**——segment/PT 需要“类型 + 长度”唯一，合并阶段必须串行写入同一哈希表；
- (b) **高冲突锁竞争**——频繁对共享频率计数加锁，尤其在短片段上形成热点，抵消前端切割并行收益；
- (c) **缓存局部性恶化**——线程分割结果随机化了后续插入顺序，增加了哈希表扩容与重排成本。

因此，**高度依赖的统计管线更适宜单线程执行或采用细粒度策略**：诸如为每个线程维护本地直方图后再分批归约、使用无锁并发字典或将 segment 空间分桶分区等。在本报告的硬件与规模下，上述改造尚未显著优于纯串行基线，但为未来大规模数据集提供了可行的优化方向。

最终，SIMD $\oplus$ OpenMP 在 01 等级取得全流程 $11.15 \times$ 峰值加速，成为当前硬件与代码约束下的最优实现；后续工作应聚焦通信/内存瓶颈与算法级分批策略，以期在 GPU 与多节点场景中进一步扩大速度—准确率折中优势。